

ShopSphere

S9 - Wishlist and Save-for-Later

Backend + DB

Platform-Specific Sprint Planning Document

This document intentionally defines only the behavior and delivery scope of the selected platform. It does not mix implementation details from the other platforms.

1. Sprint Definition

Sprint: S9 - Wishlist and Save-for-Later

Platform: Backend + DB

Application: ShopSphere

Objective: Implement the backend and database boundary for Wishlist and Save-for-Later, including business rules, persistence and API behavior.

This sprint is complete only when the stated use cases, acceptance criteria and tests pass for this platform.

2. Business Use Cases

ID	Use Case	Actor	Precondition	Main Flow	Expected Result
UC-01	Add wishlist item	Customer	Authenticated and product visible	Tap wishlist	Item is stored once.
UC-02	Move wishlist item to cart	Customer	Wishlist item exists	Move item	Server revalidates stock/price and adds if valid.

3. Detailed Implementation Plan

1. Create wishlist persistence with uniqueness rules.
2. Implement add/remove/list/move-to-cart with ownership checks.

4. Data / State / Contract Scope

- wishlists
- wishlist_items

API / Integration Contract

- Feature APIs for this sprint, versioned under /api/v1.
- Every endpoint must define authentication, authorization, validation, error codes and request correlation.

5. Rules and Expected Behavior

- The platform must rely on the backend as the source of truth for business rules.
- Do not expose stack traces, secrets or internal implementation details to end users.
- Handle loading, empty, validation and error states explicitly.
- Requests that can be retried must be safe from duplicate side effects where the feature requires idempotency.
- Ownership and authorization must be checked server-side for protected data.

6. Acceptance Criteria

- Data model supports the required business relationships and constraints.
- API enforces authentication, authorization, validation and ownership where applicable.
- Business rules are server-authoritative.
- Transactions/idempotency/concurrency controls are used where the feature requires them.
- Controlled error codes and request correlation are implemented.

7. Test Cases

TC	Scenario	Expected Result
TC-S9-01	Duplicate item	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S9-02	Deleted product	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

TC	Scenario	Expected Result
TC-S9-03	Inactive variant	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S9-04	Out of stock	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S9-05	Unauthorized wishlist access	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.
TC-S9-06	Move-to-cart failure	Operation is handled according to the defined business rule; no incorrect state or security leak is produced.

8. Sprint Exit Checklist

- Implementation completed only for this platform.
- All primary and negative use cases pass.
- API contracts are implemented/consumed as defined.
- Validation and authorization tests pass.
- Loading/empty/error handling is verified where applicable.
- No critical or blocker defect remains.
- Automated tests pass in CI.
- Documentation and contract changes are committed.

9. Out of Scope for This Document

- Implementation behavior of Customer Portal. That behavior belongs in the separate S9 document for that platform.
- Implementation behavior of Admin Portal. That behavior belongs in the separate S9 document for that platform.